

Documentation: News-Driven Forecasting Framework for Office Real Estate Market Trends

Goal

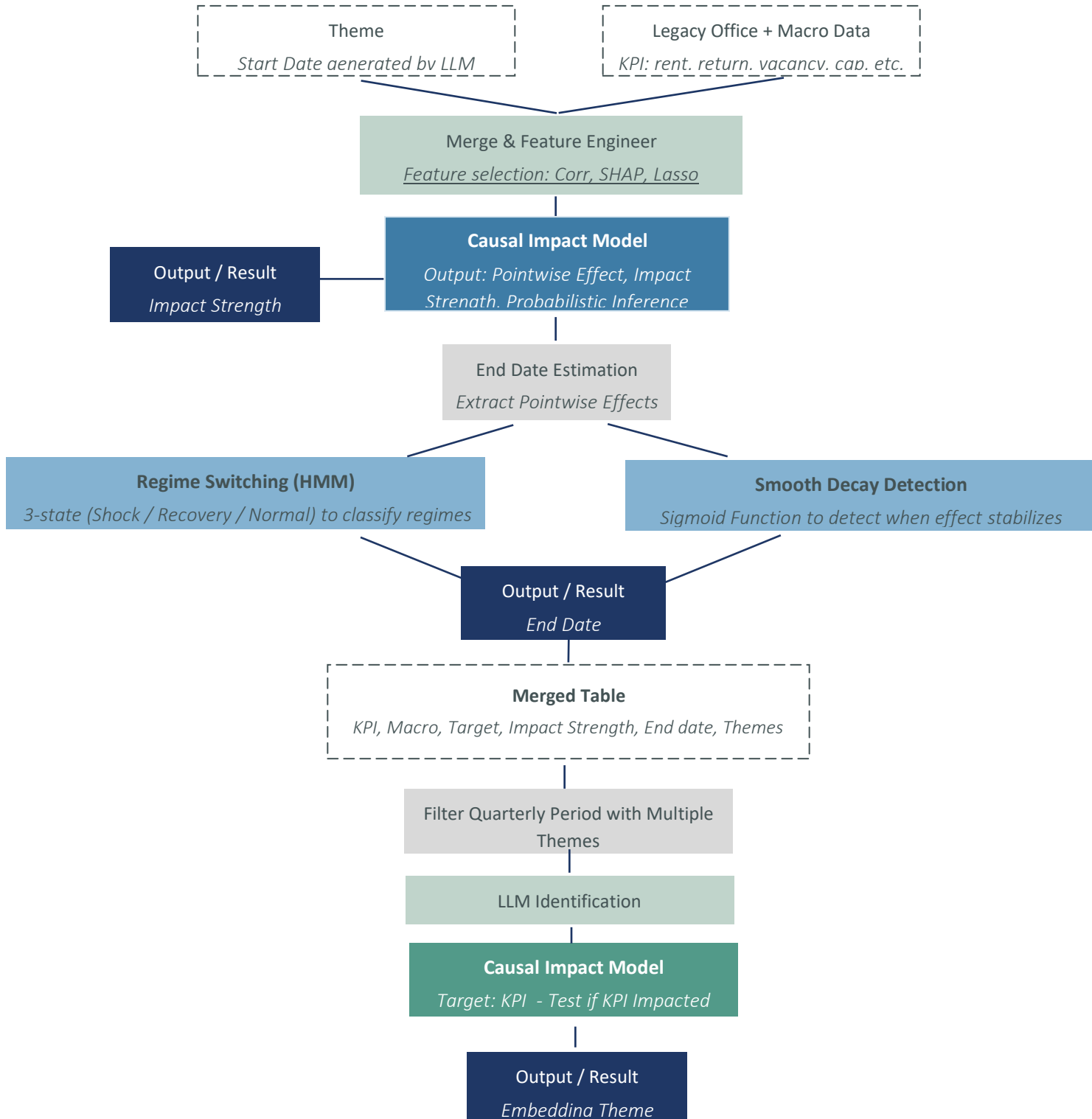
The goal of this project is to evaluate and predict how real-world themes—extracted from news sources—affect the **performance and trend of the office real estate market**, particularly focusing on sub-sectors like General Office, Medical Office, and Modern Office.

The project aims to:

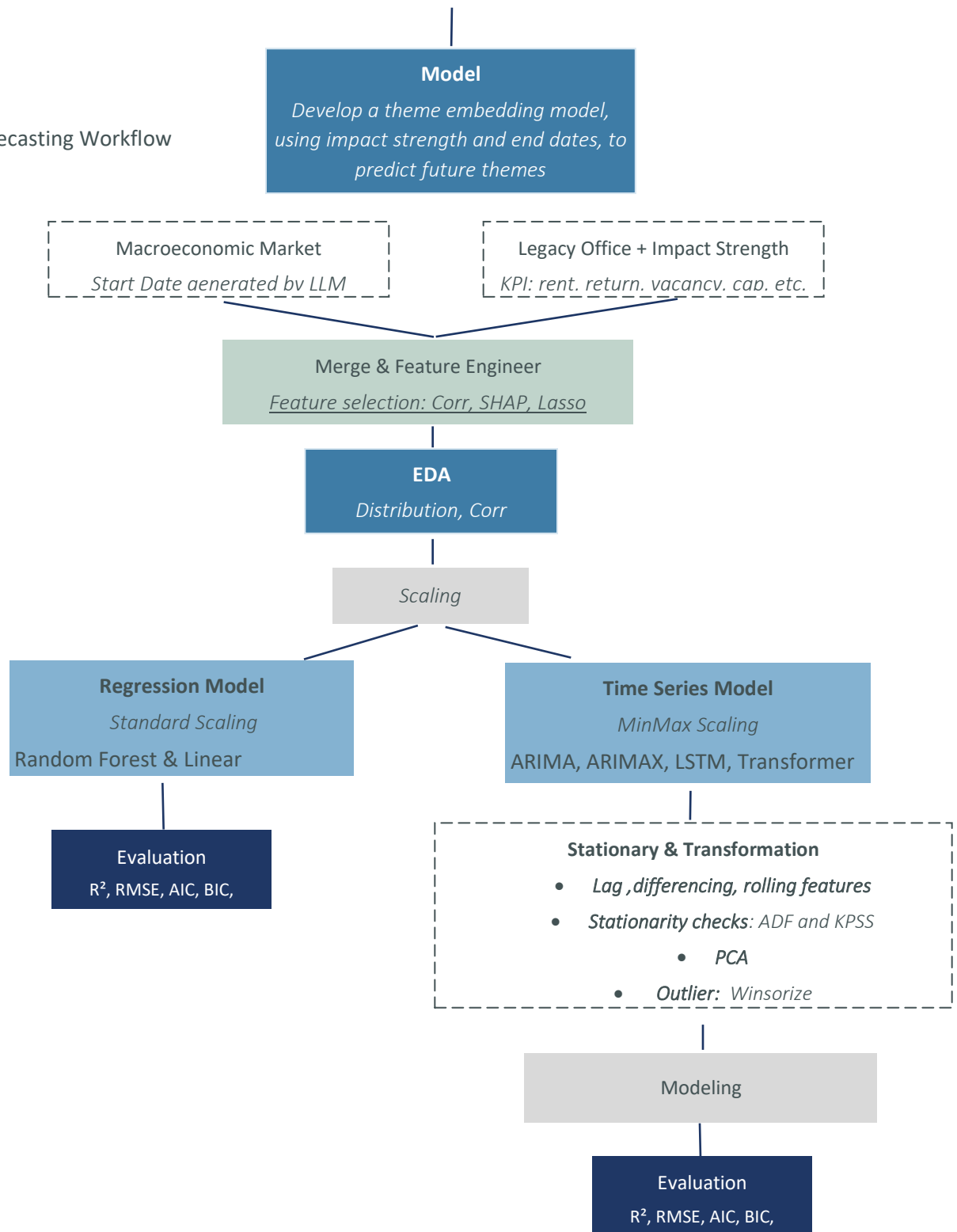
1. **Quantify the impact** of specific news-driven events;
2. **Estimate when the impact** of an event begins and ends, identifying how long the market remains in a “shock” or “recovery” regime;
3. **Forecast future trend** reversals or continuation, using historical responses to similar themes to support strategic investment decisions

Overview of Workflow

1. Causal Impact Workflow



2. Forecasting Workflow



Causal Impact Solution Directory

1. Data Collection
2. Feature Engineering
3. Causal Impact Modeling = Impact Strength + Pointwise Effect
4. Changepoint + Regime Switching Model/Smooth Decay Detection = End Date
5. Tracing Back to Exact Theme
6. Concern and Limitation
7. Next Steps

Step 1: News and Office Data Collection

a. Theme Identification from News

- **Source 1: Ata's Green Street Feed**
 - Contains curated themes generated by the LLM based on Green Street News.
 - **Limitation:** Some theme timestamps are coarse — often only at the year level, without exact quarter/month, making alignment with market data difficult.
 - **Solution:** I used an LLM to infer the likely month of impact onset based on the narrative content of the news.
- **Source 2: Aditya's Google Search API**
 - Uses automated queries to extract recent and relevant news articles.
 - **Limitation:** Although the pipeline is still under development, I've manually used the retrieved news headlines and passed them through LLMs to: Label and filter themes for use in causal modeling focusing on office sector

b. Office Data Collection

1. Sector Indicators

I used all data from ANNUALIZED_FWD_RETURNS_AND_FUNDAMENTALS and important features from each office sector from FCT_MARKET :

Legacy Office Features:

Absorption % - Trailing Year

Absorption % - Trailing Year - QoQ % Chg

Absorption % - Trailing Year - YoY % Chg

Market Value - QoQ % Chg

Market Value - YoY % Chg

Market Value - 5Y Difference

Market Value

Inventory - % Chg YoY

Inventory - % Chg YoY - YoY Difference

Medical Office Features:

NOI - Trailing Quarter

NOI - Trailing Year

NOI - Trailing Quarter - YoY % Chg

NOI - Trailing Quarter - QoQ % Chg

Income Return - Trailing Quarter - YoY % Chg

Income Return - Trailing Quarter

Income Return - Trailing Year

Modern Office Features:

Market Value - QoQ % Chg

Market Value - YoY % Chg

Market Value - 5Y Difference

Market Value

Appreciation Return - Trailing Quarter

Appreciation Return - Trailing Year

2. Macroeconomic Indicators

Tables: FCT_MARKET

Features: Vacancy Rate, Cap Rate, Rent, Absorption, Employment, Unemployment, etc.

Step 2: Feature Engineering

To identify which variables are most relevant to the office sector's performance, I applied multiple feature selection methods:

- **Correlation Analysis:** Measured linear relationships between each feature and the target.
- **Random Forest + SHAP:** Used tree-based models and SHAP values to capture non-linear feature importance.
- **L1-Regularized Regression (Lasso):** Selected a sparse set of features by penalizing irrelevant variables.

The target variable I chose is **relative sector return**, rather than absolute return.

Reason: Relative return better captures *sector-specific movement* by adjusting for broader market trends, and is more suitable when we want to isolate the effect of a specific theme on the office sector alone.

Since we're using a **Causal Impact model**, it's important to avoid **data leakage** — i.e., using features that might already embed the outcome we want to estimate.

Therefore, I avoided using features that are **strongly correlated** with the target to prevent the model from “peeking into the future” and instead selected features with **moderate or weak correlations** as supporting covariates.

Step 3: Causal Impact Model

WHY:

Among the three popular strategies in the market—**event study using REIT data, sentiment analysis with machine learning**, and the **CausalImpact model**—I chose to focus on **CausalImpact** because it is

particularly well-suited for **short-term, clean-cut events** such as policy changes, political decisions, or impactful news announcements.

CausalImpact uses a **Bayesian Structural Time Series (BSTS)** framework to simulate **counterfactual scenarios**—allowing us to estimate what would have happened without the event. This makes it possible to infer **causal relationships** from purely observational time series data, which is especially relevant for real estate markets where controlled experiments are not feasible.

In addition, CausalImpact has been widely applied in academic literature for evaluating the impact of major shocks like the **COVID-19 pandemic** and **financial market events**, proving its robustness and interpretability. Given our team's goals, I believe this method will be especially helpful, and I'm eager to explore its application in our real estate market forecasting project.

How:

1. Define the Event and Its Date
2. Select Target Variable
3. Prepare Time Series Data
4. Pre-intervention period: historical values before the event
5. Post-intervention period: values after the event
6. Build the Model
7. Interpret the Results
 - i. Point estimate: How much the target changed due to the event
 - ii. CI (confidence interval): Uncertainty range
 - iii. p-value: Was the effect statistically significant?

I define an event as having strong causal impact if it meets two conditions:

- (1) Posterior probability of a causal effect $\geq 90\%$,
- and (2) Average relative effect magnitude $\geq \pm 15\%$.

Step 4: Changing Point + Regime Switching Model/ Smooth Decay Detection = End date

1. **Extract Pointwise Effects for Change Detection**
 - Calculate the difference between actual and predicted values during the post-period to get the **pointwise treatment effects**.

- These effects reflect how much the target deviated due to the event.

2. Apply CUSUM to Detect Changepoints

- Use the **Cumulative Sum (CUSUM)** algorithm on the post-period pointwise effects to detect significant structural shifts.
- The first changepoint with recovery from the regime after the shock can be a candidate for the **impact end date**.

3. Use Hidden Markov Model (HMM) for Regime Classification

- Standardize the pointwise effect series and apply a 3-state **Gaussian HMM** to classify each time step into:
 - Shock, Recovery, or Normal regimes.
- Identify the **first transition** from Recovery → Normal as an alternative candidate for the impact end date.

4. Introduce a Rolling Post-Period Window for Robustness

- Rather than fixing a long post-period, apply a **rolling window** (12 quarters, stepping every 4).
- For each window:
 1. Re-run CausalImpact
 2. Apply HMM classification to pointwise effects
 3. Extract a potential end date if a Recovery → Normal transition occurs

5. Use Smooth Decay Detection for Comparison

To validate or compare against regime-based estimation:

1. Fit a sigmoid or exponential decay function to the post-period pointwise effect curve.
2. Define end date as the point where the fitted curve stabilizes near zero or falls below a preset threshold

Step 5: Tracing Back to Exact Theme

For each impact period, extract **multiple co-occurring themes** (e.g., "remote work", "layoffs", "interest rates").

Use an **LLM** to assess which **sector-relevant features** (e.g., vacancy rate, absorption, employment) are:

- Most correlated with the office sector's return historically

- Most closely semantically related to each theme

For each theme, return back to Causal Impact Model and set features as target to get causal impact strength:

- Did the feature shift significantly during the impact period?
- If yes, attribute the period's impact strength to this theme-feature linkage.

Step 6: Concern & Limitation

1. Impact End Date Evaluation Is Uncertain

- It's hard to validate whether the detected *end date* of a market impact is truly accurate.
- A basic approach is checking how well CausalImpact fits the synthetic counterfactual (pre-period)
- There is currently **no strong metric** to evaluate if the market has "fully recovered."

2. Theme Start Date ≠ Actual Event Date

- In practice, we use the **start of measurable market response** as the theme's start date — not the actual date the event occurred.
- This leads to ambiguity: some effects may lag behind the news significantly.
- We need to **rethink how we define the start of a theme**, possibly using lag detection or LLM-based reasoning.

3. Themes Can't Capture Long-Term Effects Automatically

- Current themes (e.g., "COVID") tend to focus on **short-term shocks** (volatility, risk repricing).
- But **second-order, long-term structural changes** (e.g., work-from-home causing lasting demand shifts in the office sector after 2024) are not captured.
- We may need to:
 - Introduce **new themes** like "Office Restructuring" to reflect lasting transformation.

4. Theme-Feature Mapping Needs Refinement

- Right now, LLMs help match themes to market features (e.g., "remote work" → vacancy rate).
- But this mapping may not capture:
 - Lag effects
 - Feature-level nonlinearity
 - Sector-specific influence strength

- We need better feature attribution logic to **explain theme** → **feature** → **return chain**.

Step 7: Next Step: Theme Embedding & Impact Prediction

Goal: To build a machine learning model that predicts the **future impact strength and duration** of a new theme, based on historical patterns across previously analyzed themes.

Step 1: Theme Embedding Construction

- Embed each finalized theme (from attribution step) into a vector representation using LLMs (e.g., OpenAI, Cohere, etc.)
- Combine each theme embedding with metadata:
 - **Impact Strength** (from CausalImpact)
 - **Impact Duration / End Date** (from HMM or Smooth Decay)
 - Optional: sector, region, source type

Step 2: Train Prediction Model

- Input: Theme embeddings + contextual features (e.g., timing, frequency, sector)
- Output:
 - Predicted **Impact Strength**
 - Predicted **End Date / Duration**
- Candidate models: Gradient Boosting (e.g., XGBoost) / Feedforward Neural Network / Siamese/Contrastive Embedding Model

Step 3: Apply to New Themes

- When new themes are extracted (e.g., via LLM + news pipeline):
 - Embed the theme
 - Use trained model to predict expected impact and duration
 - Flag high-impact upcoming themes to each sectors

Forecasting Solution Directory

1. Data Collection
2. Feature Engineering
3. Regression Models
4. Time Series Models
5. Next Steps

Step 1: Data Collection

Same dataset with CausalImpact with Impact Strength Only

Step 2: Feature Engineering

To identify which variables are most relevant to the office sector's performance, I applied multiple feature selection methods:

- **Correlation Analysis:** Measured linear relationships between each feature and the target.
- **Random Forest + SHAP:** Used tree-based models and SHAP values to capture non-linear feature importance.
- **L1-Regularized Regression (Lasso):** Selected a sparse set of features by penalizing irrelevant variables.

The target variable I chose is **relative sector return**

Features I choose:

'DATE',
'RELATIVE_RETURN',
'VACANCY RATE: CBRE IM RESEARCH',
'Permits: Residential - Multi-Family',
'Median Household Income',
'Rent-to-Income Ratio - Single-Family - 1 Year Average',
'Rent-to-Income Ratio - Single-Family',
'Households',

'Housing Cost - Payment and Maintenance',

'Home Payment - Entry-Level Home',

'Resale Listings',

'Population',

'Employment: Total',

'relative_effect_avg'

Step 3: Regression Models (Baseline)

1. Preprocessing

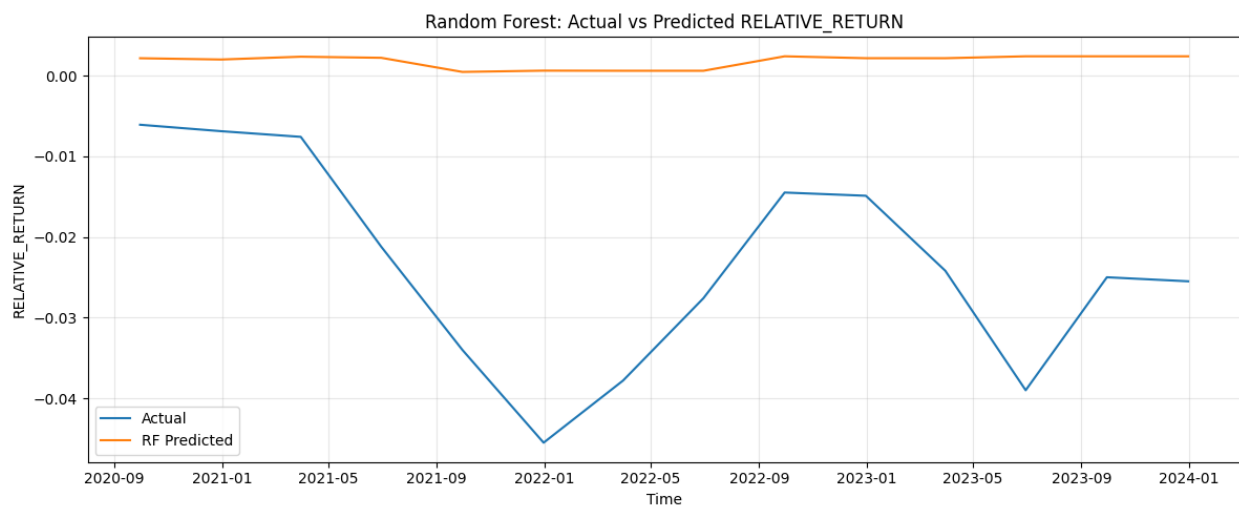
Applied **Standard Scaler** for normalization – the most basic one

2. Random Forest Model

Captures non-linear relationships and interactions between features

Evaluation Results:

- Random Forest Test MSE: 0.000777
- Random Forest Test MAE: 0.025322
- Random Forest Test R^2 : -4.2986



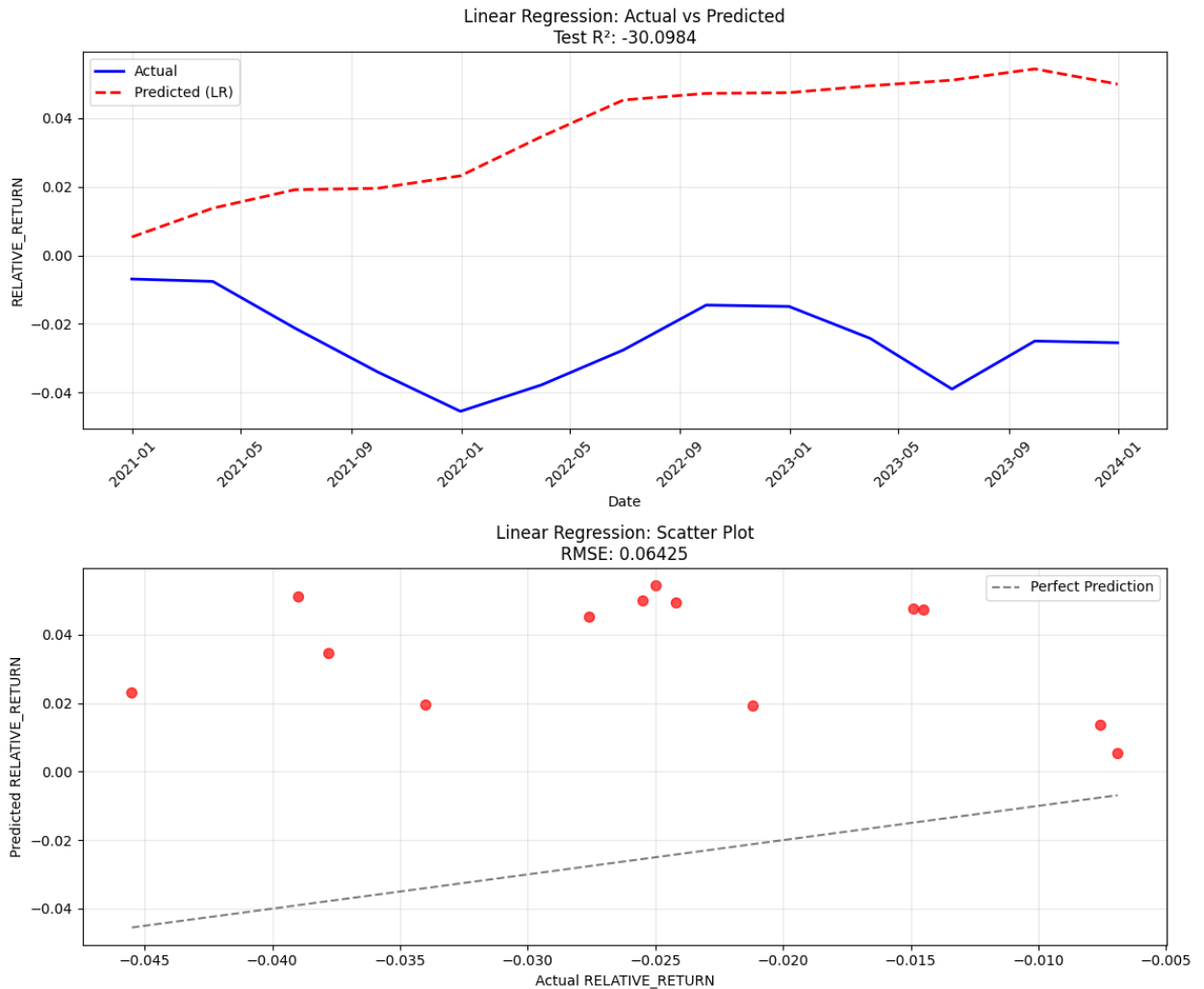
3. Linear Regression Model

A simple, interpretable baseline model assuming linear relationships

Evaluation Results:

- TRAINING METRICS:
- R^2 Score: 0.4499
- MSE: 0.0000
- MAE: 0.0027

- TEST METRICS:
- R^2 Score: -30.0984
- MSE: 0.0041
- MAE: 0.0604



Step 4: Time Series Models

1. Preprocessing

1. Data Collection & Merging

- Loaded multiple datasets (macro, sector, legacy, causal impact).
- Merged them on the DATE column to create a unified DataFrame.

2. Feature Engineering

- Selected relevant features for modeling, based on LASSO, SHAP, Correlationship
- Looked up distribution for each feature

3. Handling Outliers

- Applied winsorization to selected features to limit the influence of extreme values.

4. Scaling

- Used MinMaxScaler to standardize or normalize features

5. Stationarity Checks & Transformation

- Checked stationarity of each feature using ADF and KPSS tests.

- For non-stationary features, applied:
 - First differencing
 - Log transformation (if all values positive)
 - Log + differencing
- Chose the transformation that made the feature stationary (lowest ADF p-value).

6. Temporal Feature Engineering

- Added cyclical features for month and quarter using sine/cosine encoding to help models capture seasonality.
- Used PCA to find the best combination of features

7. Sequence Preparation

- Converted the time series data into sequences (sliding windows) for deep learning models (LSTM/Transformer).
- Used a fixed sequence length (e.g., 6 months).

8. Train/Test Split

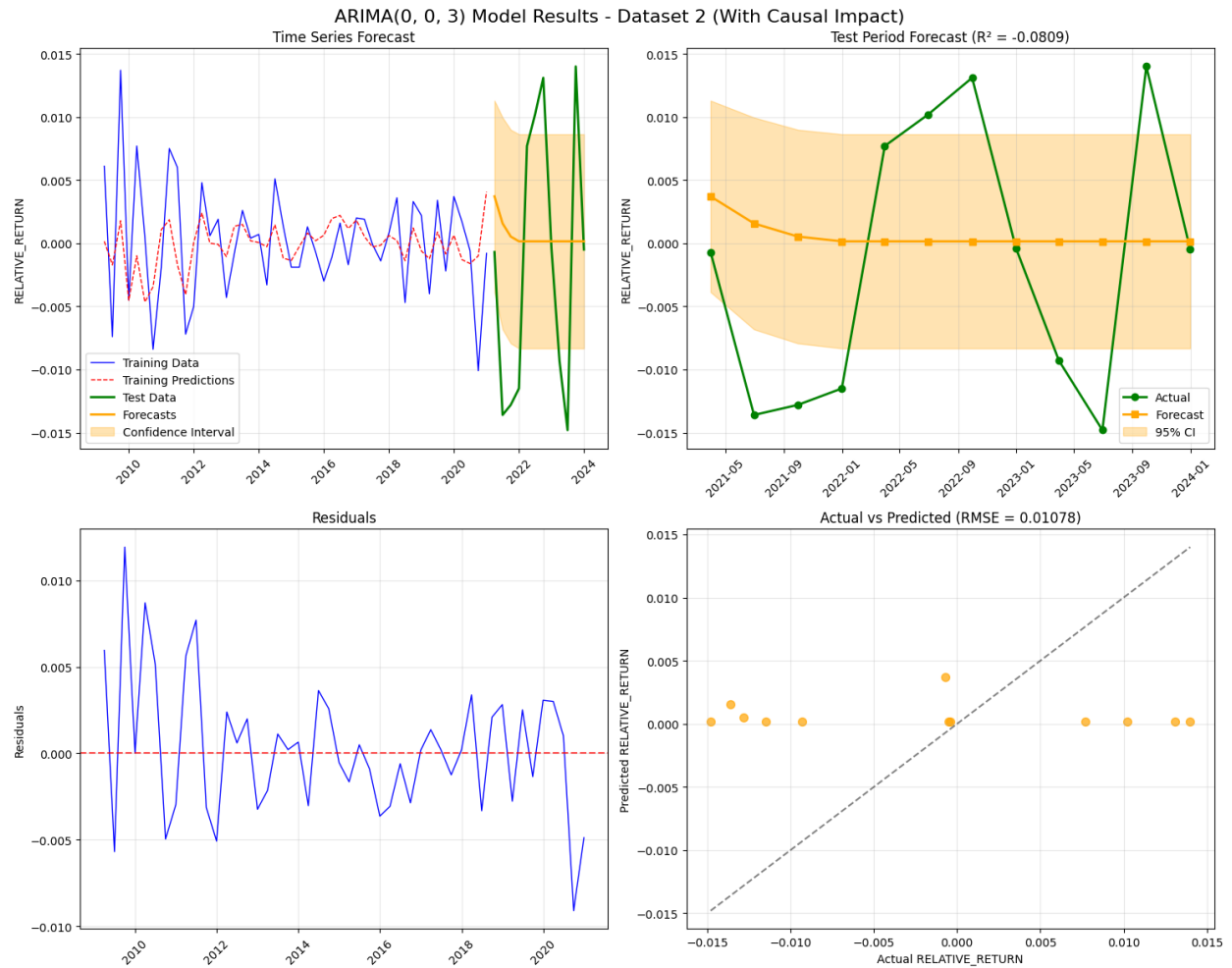
- Performed a time-based split (80% train, 20% test) to avoid lookahead bias.

2. ARIMA

Basic univariate forecasting model

Evaluation Results:

Metric	Dataset 1 (No Impact)	Dataset 2 (With Impact)
ARIMA Order	(0, 0, 3)	(0, 0, 3)
AIC	-386.5111	-386.5111
BIC	-377.1551	-377.1551
Train R ²	0.2032	0.2032
Test R ²	-0.0809	-0.0809
Test RMSE	0.010778	0.010778
Test MAE	0.009546	0.009546
Ljung-Box p-value	0.873016	0.873016



3. ARIMAX

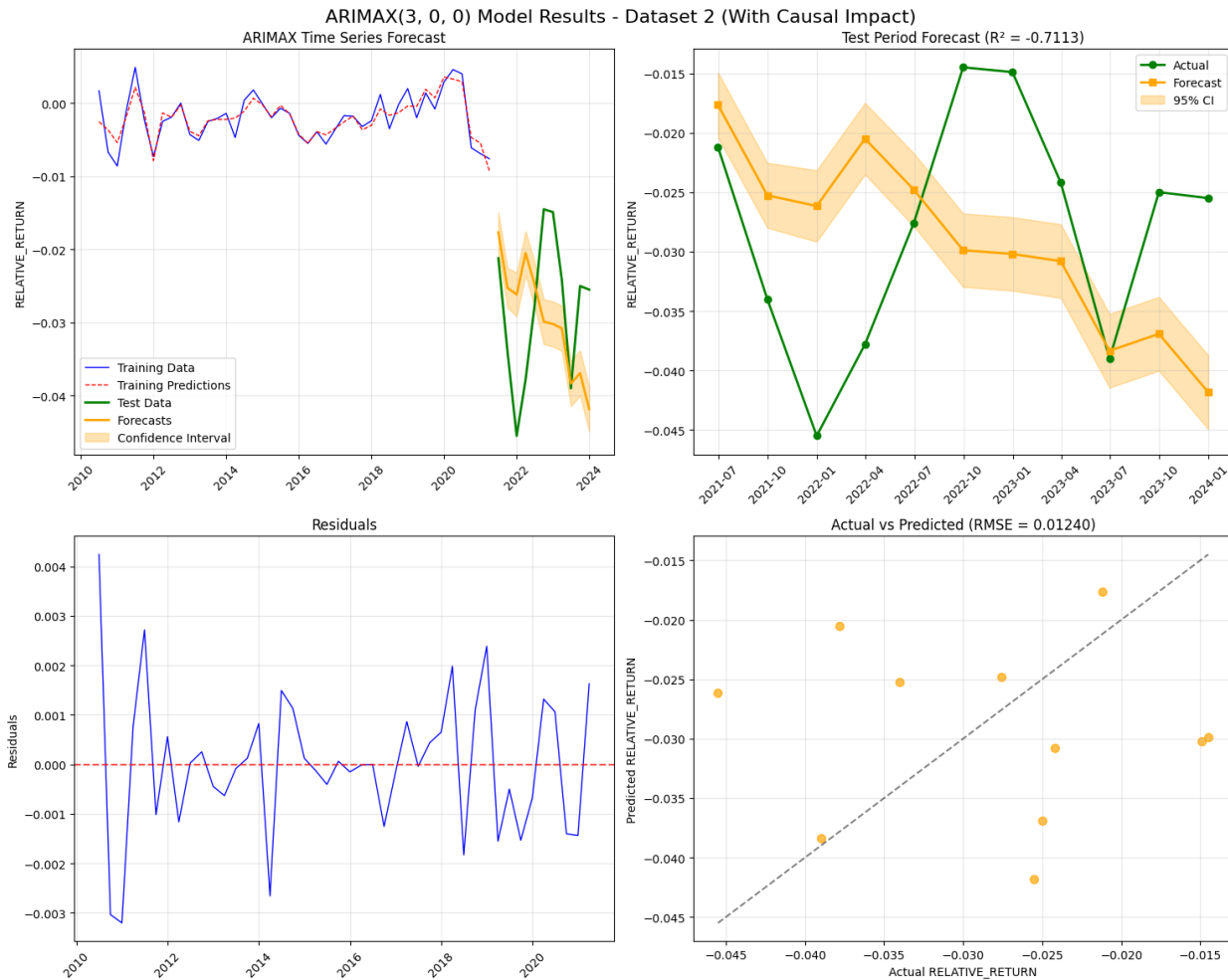
Multivariate extension using external regressors

Evaluation:

Metric Dataset 1 (No Impact) Dataset 2 (With Impact)

ARIMAX Order	(3, 0, 0)	(3, 0, 0)
Selected Features	20 features	20 features
AIC	-402.9396	-402.9396
BIC	-358.3348	-358.3348
Train R^2	0.8026	0.8026
Test R^2	-0.7113	-0.7113

Test RMSE	0.012405	0.012405
Test MAE	0.010717	0.010717
Ljung-Box p-value	0.240841	0.240841



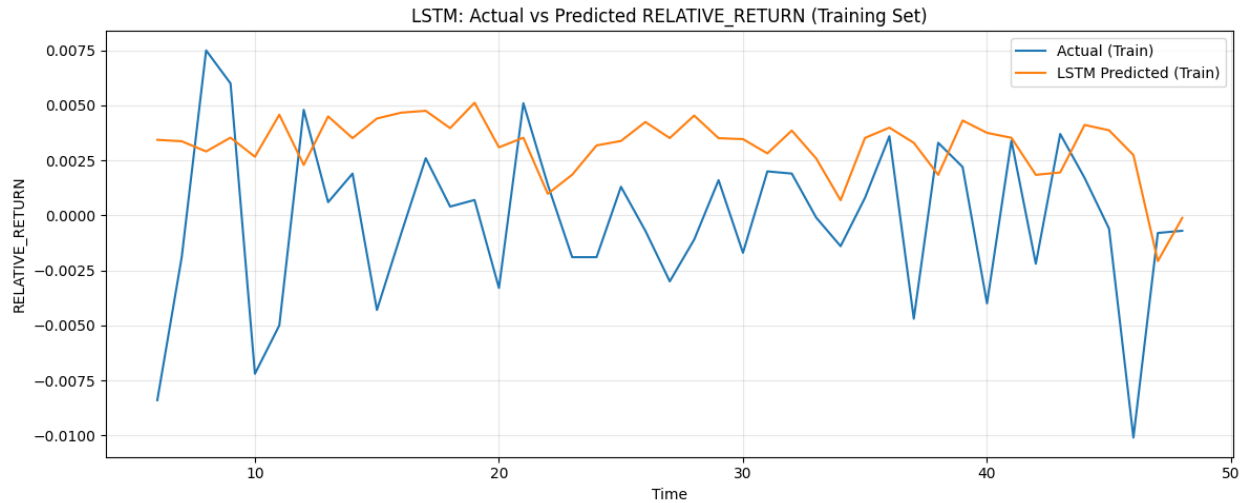
4. LSTM

Recurrent Model that captures sequential patterns

Evaluation:

LSTM Test MSE: 0.000122

LSTM Test R^2 : -0.0428



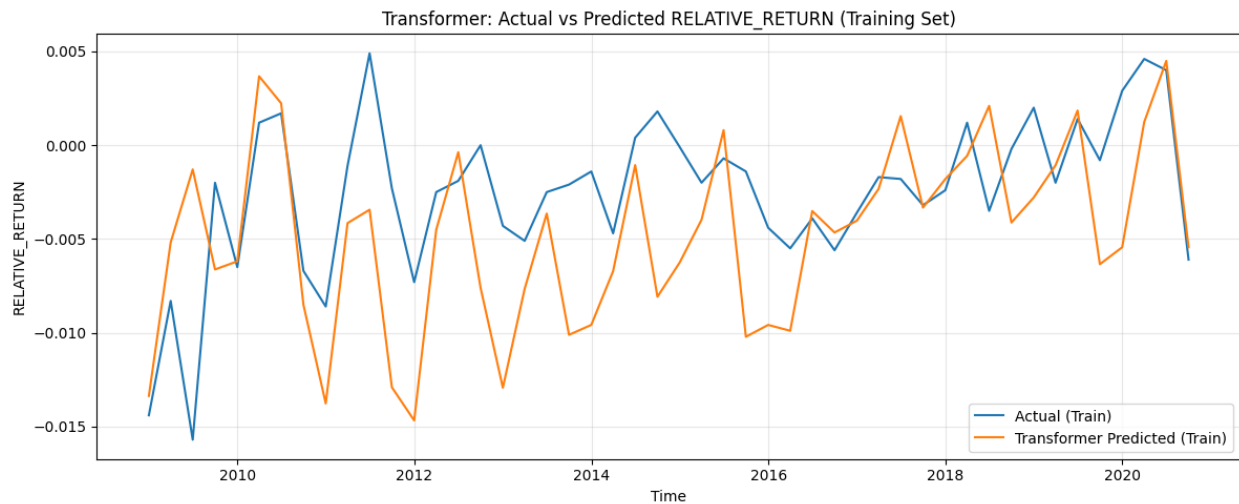
5. Transformer (BEST)

It performs the best due to ability to model long-term dependencies and complex interactions

Evaluation:

Transformer Train R^2 : -0.5643

Transformer Test R^2 : -5.7517



Step 5: Problem and Next step

1. Current Concern - Model Performance

Concern: Negative R^2 Across All Models

- All regression models, including Transformer, currently produce **negative R^2 scores**, indicating poor fit.
- **Key reason:** The available features (e.g., macro, CausalImpact outputs) likely carry **weak or noisy signals** that don't strongly explain market movement.

Solution:

1. Feature Engineering,

Feature Scaling Helps but Has Tradeoffs

- Applied **normalization and outlier handling**, which improved performance slightly.
- However, **excessive scaling** risks **removing meaningful variation** or distorting the economic meaning of some features.

2. Enrich the Dataset

- Increase the number of data points (temporal or spatial)
- Add **new feature sources**, such as:
 - Sentiment scores from news headlines
 - Thematic frequency or duration
 - Broker reports or macro indicators

3. Add Sentiment Analysis

- Run LLM-based or lexicon-based sentiment analysis on news data
- Integrate sentiment features as model inputs
- Potentially apply **Naive Bayes classification** as an additional method, which handles sentiment well and is robust on text-derived features

4. Fine-Tune the Transformer

- Test **different hyperparameters** (hidden size, learning rate, attention heads, etc.)
- Apply regularization (dropout, early stopping)
- Use cross-validation to search for setups that turn R^2 positive
- Reason for keeping Transformer:
 - Its ability to capture **weak signals across many features**
 - Avoids reliance on returns as input, **reducing data leakage risk**